A Proposal to Develop a Self-Learning Edge Language Model Based on a Weight Virtual-Memory Architecture

By: Tianrui Bai University of Wisconsin-Madison Electrical and Computer Engineering Department

Project Significance:

This proposal outlines a plan to scale and evaluate TAIS Obsidian, a self-learning large language model (LLM) architecture built around a "weight virtual memory" mechanism, from a validated 0.1B pilot to a 1B-parameter research model, and to assess its capability to learn continuously during deployment on edge-class hardware.

Pre-trained LLMs store their knowledge in frozen weights. Once deployed, a model cannot absorb new knowledge without expensive re-training. The industry-standard workaround, Retrieval-Augmented Generation (RAG) [1], and its active variants (FLARE [2], Self-RAG [3]) attach retrieved text to the prompt, but this approach has three fundamental limitations. First, plain text is not the model's native knowledge representation; knowledge is patched into the context window rather than into a weight-level interface. Second, the model has no explicit readout of its own knowledge boundary; it cannot reliably "know that it does not know," which is a root cause of hallucination [4]. Third, whatever the model learns during interaction cannot sediment into a reusable, auditable, long-term memory.

These limitations matter most where LLMs are growing fastest: the edge. The small language model (SLM) market is projected to grow from USD 0.93 billion in 2025 to USD 5.45 billion by 2032, a 28.7% CAGR, driven by on-device deployment on smartphones, IoT devices, and embedded systems that demand low latency, data control, and energy efficiency [5]. A 2026 survey of edge LLM deployment reaches the same conclusion: running models close to the data source reduces latency, enhances privacy, and conserves bandwidth, but resource constraints limit model capacity [6]. An edge model is necessarily small, and a small model necessarily has limited parameterized knowledge — so the ability to keep learning after deployment is not a luxury but the central requirement for edge LLMs.

Two further engineering barriers stand in the way. Attention's KV cache grows linearly with context length and model size, while attention computation itself scales quadratically; a single long sequence can consume gigabytes of cache memory, which is prohibitive on edge devices [7]. And naive continued fine-tuning is known to cause catastrophic forgetting — new data overwrites old weights — while recent work shows that even the choice of optimizer materially changes how much is forgotten [8].

TAIS Obsidian addresses these problems by elevating "knowledge" to a runtime object at the same level as weights — the KnowledgeBlock — and by managing knowledge blocks with an operating-system-style virtual memory: a page table (SQLite), tiered storage (L0 VRAM / L1 DRAM / L2 NVMe / L3 remote), fail-closed page faults, and read/write asymmetry in which only zero-gradient writes are allowed at runtime while gradient-based consolidation happens offline during a "sleep" phase [9]. On top of this memory system, a layered metacognition module (KAL) reads out the model's internal states to detect knowledge gaps, so the model actively retrieves or asks instead of guessing [4], [10]. The architecture has been implemented and validated at 0.1B scale: a GDN-2 linear-attention backbone with a three-level retrieval attention stack (sliding window + compressed selection + heavily compressed gist) keeps long-context cost near-linear [11], [12], and the full self-learning loop — sense, inquire, verify, write, recall, sleep-consolidate — has been demonstrated end-to-end with 437 unit tests passing.

Figure 1, the TAIS Obsidian architecture: backbone, KAL metacognition, HRL retrieval, knowledge-block library, runtime memory bus, and the sleep consolidator (see docs/TAIS_Obsidian_架构详图.png).

Objectives:

I propose to scale and rigorously evaluate the validated 0.1B pilot into a 1B-parameter self-learning research model. My goals are:

To pre-train a 1B-parameter model (1,017.7M parameters, 24 GDN-2 linear-attention layers + 8 three-level retrieval attention layers) on a 10B-token multi-domain corpus, followed by a 1B-token mid-training annealing phase with a quality-upweighted data mixture, replicating the multi-stage WSD recipe used by SmolLM2 and the Dolmino mid-training of OLMo 3 [13], [14].

To migrate and re-validate the pilot's endogenous components at 1B scale — KAL metacognition probes (knowledge-gap AUROC), HRL block retrieval (top-1 hit rate), HCA injection recall, and the honest-degradation behavior — measuring whether component strength scales with model size as the design predicts.

To extend the native context window from 1,024 tokens toward 256K tokens through RoPE cache expansion with YaRN scaling and a progressive window curriculum, the mainstream approach validated by Qwen2.5-1M and Llama 3 long-context training [15], [17].

To prepare the 1B checkpoint for HuggingFace distribution with a documented, reproducible inference path, and to benchmark the model on standard lightweight evaluations (ARC-Easy/Challenge, HellaSwag, PIQA) against same-size baselines, with all undertraining caveats explicitly reported.

This proposal builds directly on completed work: the 0.1B pilot is fully implemented and measured (Sections in the accompanying report), the 1B configuration (d_model 1536 × 32 layers, Muon optimizer) has been instantiated and verified, the 10B-token streaming data pipeline has been smoke-tested end-to-end, and the full training-to-upload toolchain has passed a 437-test regression suite.

Drafted Research Plan:

For this project, I will primarily rely on peer-reviewed machine learning venues (NeurIPS, ICLR, EMNLP, ACL), openly published technical reports (OLMo, SmolLM, Qwen, DeepSeek), and the experimental artifacts already produced by this project (training logs, evaluation reports, and 437 passing tests).

For the first objective of pre-training the 1B model, I will follow the data curriculum of OLMo 3 (Dolma 3 Mix for pre-training, Dolmino high-quality mixture for mid-training annealing) [14] and the multi-stage WSD schedule of SmolLM2 [13]. The corpus (10B tokens) mixes FineWeb-Edu (73%), mathematics (NuminaMath-CoT plus FineMath-4+, 12%), synthetic textbooks (Cosmopedia, 10%), and Chinese web text (FineWeb2-HQ, 5%). I acknowledge that 10B tokens is half of the Chinchilla compute-optimal amount for 1B [16] and far below current practice (4T+ tokens for 1B-class models [13]); this run is positioned as an architecture-validation pilot, and absolute capability will be reported as such.

For the second objective of component migration, I will reuse the pilot's validated pipelines: the KAL truth-anchor calibration procedure that reached AUROC 0.845/0.829 on two evaluation protocols at 0.1B, the HRL indexer training that reached top-1 retrieval of 1.000, and the gated fusion injection that reached 0.625 recall against an in-context upper bound of 0.70. Each metric will be re-measured at 1B to test the design's scaling predictions.

For the third objective of context extension, I will apply YaRN-based RoPE scaling with a progressive curriculum (4K → 16K → 64K → 256K), following the staged approach of Llama 3 (six stages from 8K to 128K) [17] and the YaRN-plus-sparse-attention path of Qwen2.5-1M [15]. The three-level retrieval attention already confines exact attention to a 512-token sliding window, so the RoPE load is limited to that branch; compressed selection and gist branches are position-free by design [12].

For the final objective of distribution and benchmarking, I will package the checkpoint with its tokenizer and a model card that honestly labels the undertrained research-pilot status, and I will run the lightweight evaluation suite used for 1B-class models. To make the checkpoint loadable by standard tooling, I will add an auto_map/trust_remote_code export path as follow-up engineering.

Project Motivation:

I am a senior in the Electrical and Computer Engineering Department at UW-Madison with a strong interest in the intersection of computer architecture and machine learning systems. This project began from a simple observation: operating systems solved the problem of limited fast memory a long time ago with virtual memory and page tables, but language models still have no equivalent mechanism for knowledge. I designed TAIS Obsidian to test whether that analogy can be made literal — knowledge paged in and out of a running model in weight space, with the model itself aware of what it does not know. Over the past months I implemented the full architecture from scratch in pure PyTorch, validated every subsystem on a 0.1B pilot, and documented both positive and negative results honestly. This proposal is the next step: showing that the ideas survive contact with a realistic model size and a realistic training budget.

Reference:

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020, arXiv:2005.11401.

[2] Z. Jiang, F. F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, and G. Neubig, "Active retrieval augmented generation (FLARE)," in *Proc. EMNLP*, 2023, arXiv:2305.06983.

[3] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *Proc. ICLR*, 2024, arXiv:2310.11511.

[4] A. Azaria and T. Mitchell, "The internal state of an LLM knows when it's lying (SAPLMA)," in *Findings of EMNLP*, 2023, arXiv:2304.13734.

[5] MarketsandMarkets, "Small language model market report 2025–2032," 2025. [Online]. Available: https://www.marketsandmarkets.com/Market-Reports/small-language-model-market-4008452.html

[6] E. Kristiani, V. K. Verma, and C.-T. Yang, "Deploying LLM transformer on edge computing devices: A survey of strategies, challenges, and future directions," *AI*, vol. 7, no. 1, p. 15, Jan. 2026.

[7] "KV cache optimization strategies for scalable and efficient LLM inference," arXiv:2603.20397, 2026.

[8] Y. Liu, "Optimizer-model consistency: Full finetuning with the same optimizer as pretraining forgets less," arXiv:2605.06654, 2026.

[9] J. L. McClelland, B. L. McNaughton, and R. C. O'Reilly, "Why there are complementary learning systems in the hippocampus and neocortex," *Psychological Review*, vol. 102, no. 3, pp. 419–457, 1995.

[10] "Hallucination is linearly decodable from mid-layer hidden states in quantized LLMs," arXiv:2606.02628, 2026.

[11] A. Hatamizadeh, Y. Choi, and J. Kautz, "Gated DeltaNet-2: Decoupling erase and write in linear attention," NVIDIA, arXiv:2605.22791, 2026.

[12] J. Yuan *et al.*, "Native sparse attention: Hardware-aligned and natively trainable sparse attention," DeepSeek-AI, arXiv:2502.11089, 2025.

[13] L. Ben Allal, A. Lozhkov, E. Bakouch, G. Martín Blázquez, G. Penedo, *et al.*, "SmolLM2: When smol goes big — Data-centric training of a small language model," arXiv:2502.02737, 2025.

[14] OLMo Team, "OLMo 3: Fully open language models," Allen Institute for AI, arXiv:2512.13961, 2025.

[15] Qwen Team, "Qwen2.5-1M technical report," arXiv:2501.15383, 2025.

[16] J. Hoffmann, S. Borgeaud, A. Mensch, *et al.*, "Training compute-optimal large language models (Chinchilla)," in *Proc. NeurIPS*, 2022, arXiv:2203.15556.

[17] Llama Team, "The Llama 3 herd of models," Meta AI, arXiv:2407.21783, 2024.